

# Asymptotic

## Correctness, Contracts, and Engineering Audit

Comparison with built-in Wolfram Language capabilities,  
source-derived findings, and a development plan

|                  |                                                |
|------------------|------------------------------------------------|
| Repository       | VladimirReshetnikov/Asymptotic                 |
| Reviewed version | 1.8.0                                          |
| Pinned revision  | 07a9781212beb2eeb9ff16aa625b50ac27974078       |
| Review date      | 9 September 2026                               |
| Deliverable      | Article, proposed patches, tests, and evidence |

**Evidence boundary.** This is a source audit with independently executed mathematical and Python-tool checks. The connected native Wolfram service returned errors, and no local native kernel was available. Consequently, this review does *not* claim to have executed the package, reproduced its reported Wolfram tests, measured its runtime, or validated the proposed Wolfram changes in a native kernel. Native test and benchmark harnesses are included for that purpose.

## Abstract

The repository is a substantial real-asymptotic computation library, not merely a wrapper around `InverseSeries`. Its strongest contribution is an explicit computational representation of scales, branches, complete coefficient blocks, transported remainders, and reusable inverse computations. It complements Wolfram Language’s considerably broader symbolic-analysis system. This review identifies two concrete hazards in the optional `SeriesData` export path, an observable-power option precedence problem, and a source-derived example of under-refinement caused by a fixed precision margin. It also examines expensive canonicalization, resource budgeting, native-series ingestion, validation provenance, certificate scope, API consistency, and migration costs. The recommendations prioritize preserving mathematical contracts and making achieved precision observable before adding more function families. The accompanying artifacts separate executed independent checks from unexecuted native regression tests and conservative patch proposals.

## Contents

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive assessment</b>                                          | <b>4</b>  |
| <b>2</b> | <b>Snapshot, method, and limits of this audit</b>                    | <b>4</b>  |
| 2.1      | The object actually reviewed . . . . .                               | 4         |
| 2.2      | Evidence levels . . . . .                                            | 5         |
| 2.3      | Inspection and execution scope . . . . .                             | 5         |
| 2.4      | Current upstream validation is substantial but scoped . . . . .      | 6         |
| <b>3</b> | <b>What the package implements</b>                                   | <b>6</b>  |
| 3.1      | A scale-aware result rather than only a formula . . . . .            | 6         |
| 3.2      | Endpoint and branch normalization . . . . .                          | 6         |
| 3.3      | Implemented scale families and their boundaries . . . . .            | 7         |
| 3.4      | Reusable computation is already present . . . . .                    | 7         |
| <b>4</b> | <b>Comparison with built-in Wolfram Language</b>                     | <b>8</b>  |
| 4.1      | The correct comparison is complementary, not categorical . . . . .   | 8         |
| 4.2      | Capability matrix . . . . .                                          | 8         |
| 4.3      | Representation versus mathematical ability . . . . .                 | 9         |
| 4.4      | Order and term counts must be normalized before comparison . . . . . | 9         |
| 4.5      | Recommended user workflow . . . . .                                  | 10        |
| <b>5</b> | <b>Mathematical contracts that the software must preserve</b>        | <b>10</b> |
| 5.1      | Ordinary inverse model and complete coefficients . . . . .           | 10        |
| 5.2      | Truncation is about complete layers . . . . .                        | 11        |
| 5.3      | Remainders have more structure than an exponent . . . . .            | 11        |
| 5.4      | Transporting source error through an inverse . . . . .               | 11        |
| 5.5      | Differentiation, exponentiation, and conditioning . . . . .          | 11        |

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>6 Findings registry and priorities</b>                                         | <b>12</b> |
| <b>7 Detailed defects and reproducible contract examples</b>                      | <b>13</b> |
| 7.1 A01: A sparse answer triggers a dense allocation . . . . .                    | 13        |
| 7.2 A02: Logarithmic remainder information disappears on export . . . . .         | 14        |
| 7.3 A03: Explicit observable powers are silently shadowed . . . . .               | 14        |
| 7.4 A04: Refinement can return a valid result below its requested goal . . . . .  | 15        |
| 7.5 A05: Normalization hides an integer-factorization problem . . . . .           | 16        |
| <b>8 Audit of mathematical and implementation invariants</b>                      | <b>16</b> |
| 8.1 Sparse ordering, resonances, and the comparator candidate . . . . .           | 16        |
| 8.2 Native-series ingestion is an analytic boundary . . . . .                     | 17        |
| 8.3 Fourier coefficients and oscillatory zeros . . . . .                          | 17        |
| 8.4 Finite flat sectors are not a universal transseries field . . . . .           | 18        |
| 8.5 What the exact interval certificate proves . . . . .                          | 18        |
| 8.6 Distinguishing three kinds of residual evidence . . . . .                     | 18        |
| <b>9 Performance analysis and resource design</b>                                 | <b>19</b> |
| 9.1 The relevant complexity parameters . . . . .                                  | 19        |
| 9.2 A06: one option is not one budget . . . . .                                   | 19        |
| 9.3 A16: candidates for improvement, not measured speedups . . . . .              | 19        |
| 9.4 Benchmark protocol and the supplied harness . . . . .                         | 20        |
| <b>10 API and user-experience redesign</b>                                        | <b>21</b> |
| 10.1 A09: make the order request a typed mathematical object . . . . .            | 21        |
| 10.2 A10: keep approximate evaluation distinct from checked evaluation . . . . .  | 22        |
| 10.3 A11: version the result schema and expose capabilities . . . . .             | 22        |
| 10.4 A12: normalize native conventions without hiding differences . . . . .       | 22        |
| 10.5 A13–A14: certificate discovery and distribution polish . . . . .             | 23        |
| <b>11 Testing, CI, and release evidence</b>                                       | <b>23</b> |
| 11.1 A07: packaging CI is not mathematical CI . . . . .                           | 23        |
| 11.2 A08: bring the full-suite runner up to the focused-runner standard . . . . . | 24        |
| 11.3 The highest-value new test classes . . . . .                                 | 24        |
| 11.4 What was actually run for this report . . . . .                              | 24        |
| <b>12 Proposed changes and use of the accompanying artifacts</b>                  | <b>25</b> |
| 12.1 A deliberately small containment patch . . . . .                             | 25        |
| 12.2 Helpers and native test organization . . . . .                               | 26        |
| 12.3 Independent reproduction and document build . . . . .                        | 26        |

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| <b>13 Development roadmap with acceptance gates</b>                                  | <b>27</b> |
| 13.1 Stage 1: preserve existing information and bound representation costs . . . . . | 27        |
| 13.2 Stage 2: make requests and results comparable . . . . .                         | 27        |
| 13.3 Stage 3: separate planning, algebra, evidence, and presentation . . . . .       | 27        |
| 13.4 Stage 4: optimize where measurements identify bottlenecks . . . . .             | 27        |
| 13.5 Stage 5: expand analytic scope with explicit proof obligations . . . . .        | 28        |
| <b>14 Longer-term mathematical opportunities</b>                                     | <b>28</b> |
| 14.1 Certified special-function inverses . . . . .                                   | 28        |
| 14.2 Uniformity in parameters and transition models . . . . .                        | 28        |
| 14.3 Complex sectors and branch geometry . . . . .                                   | 28        |
| 14.4 Multivariate implicit inversion . . . . .                                       | 29        |
| 14.5 Machine-checkable local evidence . . . . .                                      | 29        |
| <b>15 Conclusion</b>                                                                 | <b>29</b> |
| <b>A Source coverage and reproducibility map</b>                                     | <b>30</b> |
| <b>B Artifact inventory and status</b>                                               | <b>31</b> |
| <b>References</b>                                                                    | <b>32</b> |

# 1 Executive assessment

**The project is worth developing as a contract-aware layer above the Wolfram kernel.** Its value is not that Wolfram Language has no asymptotics, no fractional series, or no inverse expansion machinery. Those claims would be inaccurate. Its value is that it packages a particular mathematical workflow: select a real germ, work in an explicit scale, compute complete blocks, preserve an error model, and retain enough information to inspect or refine the result. The source implements ordinary power-log jets, logarithmic and exact-core reductions, Fourier-polynomial coefficients, finite flat sectors, special-function adapters, arithmetic, refinement, and several distinct kinds of validation.[1, 2, 3, 6, 8]

The library also contains safeguards that should not be sacrificed for apparent convenience. It does not equate a small numerical residual with a proof. The interval certificate explicitly concerns a stored, explicit function on a checked interval, rather than every function compatible with an unspecified asymptotic remainder. Differentiation requires a derivative contract. Fourier inverses reject an oscillatory leading block rather than inventing an eventual sign. Special-function input models distinguish finite Poincare information from convergent exact data.[9, 3, 6, 10]

The highest-priority findings concern *interfaces between otherwise reasonable components*:

1. A sparse expansion is eagerly converted to a dense rational-lattice `SeriesData`. Two retained blocks can request a billion coefficient slots. Neither the sparse term count nor the user's "MaxTerms" budget bounds this allocation.
2. Both classical-series exporters omit the logarithmic degree of the remainder. The rich object can correctly describe  $\mathcal{O}(x|\log x|)$  while the exported object no longer carries that contract.
3. The inverse-object coefficient wrapper prepends an inherited "Power" option before explicit caller options. The explicit setting is therefore silently shadowed.
4. Derived-operation refinement uses a fixed guard margin rather than propagating precision demands through valuations. A large pole supplies a simple example in which a valid returned approximation can remain far below the requested precision.

The first three findings are visible directly in the relevant source paths; the fourth includes an explicit mathematical trace, but all native reproductions remain pending. Detailed classifications and examples appear in Sections 6–7.[2, 3, 33]

The development order should be: contain the export hazards; make native validation a release gate; introduce explicit achieved-precision and capability descriptions; improve operation-level precision planning; then expand analytic scope. Replacing the mature kernel's general symbolic solvers would be a distraction. Adapting their output into carefully checked contracts is a more credible strategy.

## 2 Snapshot, method, and limits of this audit

### 2.1 The object actually reviewed

All repository conclusions refer to revision 07a9781212beb2eeb9ff16aa625b50ac27974078. The paclet manifest calls the package `AsymptoticInverse`, gives version 1.8.0, and requires `WolframVersion -> "15.0+"`. The tree contains 38 kernel `.wl` modules, in addition to `init.m`. The repository-root standalone `AsymptoticInverse.wl` is a generated distribution; the modular kernel is canonical. Historical reports and their separate implementations are not alternative current production entry points.[20, 12, 15]

This distinction matters. A defect in an archived proposal is not a defect in the maintained package. Conversely, a successful historical manifest does not establish correctness of every current module. The report therefore uses pinned source links and function names, not moving `main` links, for its implementation findings. The archive contains a module inventory, a source-coverage manifest, and an explicit native-execution status file.

## 2.2 Evidence levels

| Label | Meaning                     | What it does and does not establish                                                                                     |
|-------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| S     | Source observation          | A control path, option list, allocation, or data field is present in the reviewed text. This is not a measured runtime. |
| D     | Mathematical derivation     | A stated consequence follows from an explicit model or contract. Its assumptions are supplied.                          |
| E     | Executed independent check  | The supplied Python/SymPy or Python-tool check was run in this environment. It does not execute the Wolfram package.    |
| R     | Repository-reported result  | An upstream validation record is being reported, not independently reproduced.                                          |
| U     | Native reproduction pending | A Wolfram test or benchmark is supplied, but no native result is claimed.                                               |
| P     | Proposal or risk            | A recommended design, engineering risk, or audit candidate, not an established wrong public answer.                     |

A single finding can have several labels. For example, the dense allocation has a source-observed allocation, an exact derivation of its length, an independently executed size calculation, and an unexecuted native fixture. This is stronger than a vague performance suspicion, but it still is not a measured memory benchmark.

## 2.3 Inspection and execution scope

The audit followed the public entry points through the core jet algebra, forward and inverse construction, result formatting, classical-series export, residuals, coefficient queries, series arithmetic and refinement, source-coordinate reconstruction, Fourier inverses, native special-function ingestion, Gamma inverse construction, and elementary interval certificates. Packaging metadata, the CI workflow, the full-suite runner, development notes, and current validation summaries were also inspected. Some other family modules were inventoried and assessed through their dispatch paths and repository descriptions rather than exhaustively line-audited. Appendix A records that distinction.

The native Wolfram connector’s context and evaluation calls returned service failures. No native Wolfram executable was present locally. The independent mathematical checks ran with Python 3.13.5 and SymPy 1.14.0. There are 16 successful mathematical checks and seven successful Python-tool tests in the included evidence. These counts must not be added to the repository’s native pass counts as though they were one suite.

## 2.4 Current upstream validation is substantial but scoped

The latest 1.8.0 section of the validation record reports 305 passing tests from an explicitly selected group of 18 files, zero failures, builder and documentation checks, and isolated standalone acceptance checks. It also explicitly says that the full package suite was not run at that milestone. Its operation timings compare specific earlier and later package fixtures; they are not comparisons with Wolfram built-ins.[13]

The older `review-manifest.json` records a different baseline commit, four kernel files, and 130 successes. It is a historical artifact, not a current all-module attestation. The development documentation already warns readers to preserve that historical scope. The recommendation here is to make the current release attestation equally easy to identify and verify, not to erase the historical record.[14, 15]

## 3 What the package implements

### 3.1 A scale-aware result rather than only a formula

The central public head is `GeneralizedSeries`. An object retains a finite expression together with properties such as its variable, approach, terms, remainder, and provenance; specialized kinds add different data. `Normal[s]` deliberately returns only the finite expression. Standard and traditional formatting hide the head but retain the original object inside an interpretation. The formatting code uses held forms and read-only interpretation boxes to avoid executing payloads or retaining stale metadata after a visual coefficient edit. These are useful notebook-safety choices.[2]

The ordinary computational representation is a sparse jet

$$J = (T, \rho, d), \quad T = \{(\alpha_j, P_j(L))\}, \quad L = \log u,$$

representing a retained sum of  $u^{\alpha_j} P_j(\log u)$  and an error class

$$\mathcal{O}\left(u^\rho(1 + |\log u|)^d\right), \quad u \downarrow 0.$$

An infinite precision marker denotes absence of an unknown tail in the internal representation. Equal weights must be merged before a block is counted or a truncation frontier is selected. The code canonically handles rational and algebraic weights, attempts exact treatment of further real constants, and refuses an undecidable order rather than intentionally sorting approximate exponents.[2]

### 3.2 Endpoint and branch normalization

For the ordinary real approaches, the local coordinate is one of

$$u = x - x_0, \quad u = x_0 - x, \quad u = 1/x, \quad u = -1/x.$$

Each is positive on the selected side and tends to zero. This allows one local algebra to serve finite endpoints and both infinities. Additional source-coordinate modules recognize certain exponential and logarithmic charts, reconstruct the requested source observable, and reject an input remainder that has not first been transported into the chart. They also retain exact chart inverses when exponentiating an inexact intermediate would amplify its error unacceptably.[2, 8]

A subtle but important API convention is that a nonunit **"Power"** at a finite source endpoint describes a local source observable. The formula is not automatically the global power  $x^r$  after

an arbitrary translation. The source code’s coordinate sign and endpoint reconstruction must therefore remain part of any coefficient or refinement API. A generic wrapper that forgets this distinction can be wrong even when its formal series algebra is correct.[2, 5]

### 3.3 Implemented scale families and their boundaries

| Area                               | Existing capability                                                                                               | Important boundary                                                                                            |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Ordinary power-log                 | Exact sparse weights, logarithmic-polynomial blocks, Lagrange/Newton-related inversion, arithmetic and refinement | Not every symbolic exponent order is decidable.                                                               |
| Lambert and logarithmic reductions | Leading logarithmic/exponential cores and logarithmic coordinate scales                                           | Cutoff meanings differ from an ordinary target-power cutoff.                                                  |
| Fourier coefficients               | Finite modes in the logarithmic coordinate, complete frequency grouping and bounded coefficient envelopes         | An oscillatory leading core is not given an unproved eventual sign.                                           |
| Flat and exponential structure     | Finite flat sectors and operations, retained exponential carriers, and composite error envelopes                  | Not a claim of a universal transseries or resurgent-analysis engine.                                          |
| Gamma and Barnes families          | Dedicated positive-argument asymptotic models, Lambert-based inverse cores, residual and numerical checks         | Finite Poincare information is not convergent exact input.                                                    |
| Other special functions            | Explicit adapters plus native structured-series ingestion, parameter and real-domain checks                       | Recognizing a function head does not prove support for every parameter and endpoint.                          |
| Certificates                       | Elementary exact rational interval root enclosures for admitted inverse kinds                                     | Not all specialized inverse kinds, arbitrary special-function expressions, or unspecified remainder families. |

This inventory is grounded in the current public description and the cited implementation paths. It would be misleading to propose Fourier support, Gamma inverses, finite flat-sector arithmetic, or retained refinement state as if they were absent. The opportunities lie in making their contracts more uniform and extending their interoperability.[1, 6, 7, 11, 10, 9, 5]

### 3.4 Reusable computation is already present

The refinement implementation retains coefficient prefixes and boundaries or Newton unit jets, verifies compatibility, and includes a signature of the model, observable, assumptions, function, variables, endpoint, and direction. It uses value-like retained states rather than a global cache. Replayed operations are explicitly distinguished from reused computations in `RefinementStatistics`. These are good foundations for both performance and reproducibility. The remaining issue is not simply “add caching”; it is making precision planning, cache accounting, and replay coverage as systematic as the stored-state machinery.[5, 3]



## 4 Comparison with built-in Wolfram Language

### 4.1 The correct comparison is complementary, not categorical

Wolfram’s documented `Asymptotic` supports more than elementary power series: it infers scales, accepts finite or infinite real or complex expansion points, and can operate on expressions involving integrals, transforms, and differential-equation solutions. Its options include assumptions, conditions, precision and performance controls, and `SeriesTermGoal`. Accordingly, inability of one `SeriesData` representation to encode a particular sparse scale does not establish that Wolfram Language cannot compute any approximation in that scale. [21]

`AsymptoticSolve` is a particularly relevant baseline. It supports implicit equations and systems, can select solutions passing through a specified point, has a real-valued-solution form, and documents real and complex approach directions. Its approximation order is not generally the polynomial degree. The package’s inverse routines address a narrower but more inspectable workflow, rather than replacing all of these facilities. [22]

### 4.2 Capability matrix

| Task                          | Native baseline                                            | Package contribution or limitation                                                                                                          |
|-------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Taylor, Laurent, and Puiseux  | <code>Series</code> , <code>SeriesData</code>              | Sparse explicit blocks and richer branch/remainder provenance; native tools are the first baseline on conventional cases.                   |
| Ordinary reversion            | <code>InverseSeries</code>                                 | Complete power-log and exact weight model machinery, observable powers, and reusable computation.                                           |
| Implicit asymptotic solutions | <code>AsymptoticSolve</code>                               | Specialized single-real-branch inverse construction with retained model and checks; less general than native systems solving.               |
| Composition                   | <code>ComposeSeries</code> and series arithmetic           | Explicit transport of heterogeneous precision and carrier information in admitted scales.                                                   |
| Coefficient extraction        | <code>SeriesCoefficient</code>                             | Model multi-index coefficients and Fourier-specific queries; these are not interchangeable with coefficients indexed only by a final power. |
| General asymptotics           | <code>Asymptotic</code>                                    | Durable typed results, inspection, refinement, and explicit contracts for the supported families.                                           |
| Logarithmic inversion         | <code>ProductLog</code> , symbolic solving and asymptotics | Explicit branch and scale bookkeeping around recognized cores.                                                                              |
| Integrals                     | <code>AsymptoticIntegrate</code>                           | The reviewed package is not a general replacement for asymptotic integration.                                                               |

| Task                          | Native baseline                                                       | Package contribution or limitation                                                                                                                                  |
|-------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Discrete problems             | <code>DiscreteAsymptotic</code> and related native facilities         | Dedicated inverse/continuous-germ contracts are a different focus.                                                                                                  |
| Differential equations        | <code>AsymptoticDSolveValue</code>                                    | No comparable general differential-equation solver is established by this audit.                                                                                    |
| Asymptotic comparisons        | <code>AsymptoticLess</code> and related relations                     | Explicit exponent, logarithmic-degree, envelope, and coordinate metadata can support mechanical contract comparisons.                                               |
| Certified numerical inversion | Native numerical solvers and symbolic analysis are useful ingredients | An elementary rational-interval certificate with a stated local scope is an additional package facility; it is distinct from merely calling <code>FindRoot</code> . |

Native rows refer to the corresponding official documentation; package claims refer to the inspected implementation and current overview. [23, 24, 25, 26, 27, 21, 22, 28, 29, 30, 31, 32, 1, 9]

### 4.3 Representation versus mathematical ability

`SeriesData` represents coefficients on a regular exponent lattice, with exponents determined by integer positions and a denominator. It is well suited to conventional rational-power series. A sparse collection such as  $1 + x^{\sqrt{2}} + x^{\sqrt{3}}$  has a different natural representation. Even for rational powers, a denominator can make a dense lattice inefficient. Neither observation is a blanket statement about the reach of the native `Series` or `Asymptotic` algorithms.[24, 23]

`InverseSeries` and `ComposeSeries` provide native series reversion and composition, including conventional fractional-power situations. They should be used as differential baselines wherever the input and output contracts genuinely overlap. `SeriesCoefficient` also offers direct coefficient queries from series or expressions. The package’s multi-index query has a different meaning: several model multi-indices can contribute to the same final exponent and must be added before one obtains its complete coefficient block. [25, 26, 27, 2]

### 4.4 Order and term counts must be normalized before comparison

For the ordinary package path, a cutoff  $H$  is exclusive: retain blocks with weight strictly below  $H$ . A native call such as `Series[f,{x,0,n}]` uses the native series order convention. An `AsymptoticSolve` order can count approximation stages rather than polynomial powers. Equal integers in these calls therefore do not establish equal work or equal accuracy.[2, 23, 22]

A meaningful comparison records the local coordinate, selected branch, retained expression, first omitted scale, logarithmic degree, and whether validation work was included. A two-line native expression should not be called “faster” than a package object that additionally constructs a certificate without separately accounting for that certificate. Conversely, a large metadata object is not inherently evidence of a more accurate approximation.

## 4.5 Recommended user workflow

Use native `Series` and `InverseSeries` for standard local series when they satisfy the task. Use `Asymptotic` or `AsymptoticSolve` for broad symbolic exploration and implicit problems. Use this package when complete nonstandard blocks, a recorded real branch, precision-preserving operations, or repeatable inverse refinement are the main requirement. Then use exact identities or a separately conditioned numerical check to validate the overlap. These are workflow recommendations, not measured claims of universal superiority.

# 5 Mathematical contracts that the software must preserve

## 5.1 Ordinary inverse model and complete coefficients

Consider, on a specified real germ,

$$v = au^p \left( 1 + \sum_{j=1}^m u^{\delta_j} B_j(\log u) \right), \quad a \neq 0, \quad p \neq 0, \quad \delta_j > 0, \quad (1)$$

where  $u > 0$  tends to zero and the branch satisfies  $v/a > 0$ . Set  $z = (v/a)^{1/p}$  and  $L = \log z$ . For a fixed local observable  $u^r$ , the formal expansion has the form

$$u(z)^r = z^r \sum_{\mathbf{k} \in \mathbb{N}^m} z^{\mathbf{k} \cdot \boldsymbol{\delta}} C_{\mathbf{k}}(L). \quad (2)$$

The core's Euler-operator formula, also reflected in the Fourier implementation, can be written as follows. If  $n = |\mathbf{k}| > 0$ ,  $\Delta = \mathbf{k} \cdot \boldsymbol{\delta}$ , and  $D = d/dL$ , then

$$C_{\mathbf{k}}(L) = \frac{(-1)^{n_r}}{p^n \prod_j k_j!} \prod_{q=1}^{n-1} (D + r + \Delta + qp) \prod_j B_j(L)^{k_j}, \quad C_{\mathbf{0}} = 1. \quad (3)$$

The operators commute because they are polynomials in  $D$ . This formula explains both the exact coefficient feature and why logarithmic degree must be tracked separately from power.[2, 6]

For completeness, introduce independent perturbation markers in (1). The observable version of Lagrange–Bürmann applies  $(n-1)$  target derivatives to the product of the core observable derivative and  $n$  perturbation factors. A target derivative becomes a factor  $p^{-1}z^{-p}(D + \lambda)$  when applied to  $z^\lambda P(\log z)$ . Collecting powers and selecting a multi-index yields the multinomial factor  $n! / \prod k_j!$ , which cancels the  $n!$  in the formula. The remaining factors are exactly those in (3). This derivation is formal coefficient algebra; applicability to an unknown real function still requires the branch and remainder hypotheses discussed below.

For  $f(u) = u + u^2 \log u$ , independently solving the composition residual gives

$$\begin{aligned} f^{-1}(y) = & y - y^2 L + y^3(2L^2 + L) \\ & - y^4 \left( 5L^3 + \frac{11}{2}L^2 + L \right) \\ & + y^5 \left( 14L^4 + \frac{73}{3}L^3 + \frac{21}{2}L^2 + L \right) + \cdots, \quad L = \log y. \end{aligned} \quad (4)$$

The included Python check obtains these coefficients by introducing one unknown coefficient at a time and cancelling the residual, rather than copying the package's Euler implementation. It also verifies zero residual through the displayed degree. This is an independent algebraic oracle, not an execution result from the package.

## 5.2 Truncation is about complete layers

Suppose different multi-indices satisfy  $\mathbf{k} \cdot \boldsymbol{\delta} = \mathbf{l} \cdot \boldsymbol{\delta}$ . Their contributions belong to the same block. Truncating before all such contributions are added can leave a wrong coefficient or an incorrect nonzero frontier after cancellation. Even simple algebraic descriptions can conceal equality:  $\sqrt{8} = 2\sqrt{2}$ . The ordinary and Fourier implementations already devote machinery to grouping these layers. The recommended optimizations must preserve that invariant rather than reverting to a first- $n$ -indices heuristic.[2, 6, 5]

## 5.3 Remainders have more structure than an exponent

At a fixed positive  $u$ , the notation

$$R(u) = \mathcal{O}(u^\rho(1 + |\log u|)^d)$$

asserts an eventual bound with some constant and some neighborhood. It is not itself a computable error bound at that  $u$ . It also cannot generally be replaced by  $\mathcal{O}(u^\rho)$  when  $d > 0$ . For every fixed  $\varepsilon > 0$ ,

$$u^\varepsilon(1 + |\log u|)^d \longrightarrow 0, \quad u^\rho(1 + |\log u|)^d = \mathcal{O}(u^{\rho-\varepsilon}), \quad (5)$$

but the loss of  $\varepsilon$  is real information loss. This observation is the mathematical basis for the conservative native-ingress policy examined later, and for finding A02.

## 5.4 Transporting source error through an inverse

Assume the source model has an error  $\mathcal{O}(u^\rho L_u^d)$ , where  $L_u = 1 + |\log u|$ , and that the actual derivative satisfies the matching nondegenerate branch behavior  $f'(u) \sim apu^{p-1}$ . The inverse-coordinate error is then, by local mean-value conditioning,

$$\mathcal{O}(u^{\rho-p+1} L_u^d).$$

For the observable  $u^r$ , the corresponding error is

$$\mathcal{O}(u^{\rho-p+r} L_u^d). \quad (6)$$

In the positive target error coordinate this gives exponent  $(\rho - p + r)/|p|$ . At a source infinity, the internal observable exponent is changed as required by  $x = \pm 1/u$ . This explains why increasing the number of formally inverted coefficients cannot exceed the information supplied by the source remainder.[2, 6, 5]

The derivative hypothesis is not cosmetic. An arbitrary function can have a small magnitude remainder and a very large derivative remainder. Consequently, declarations of "InputRemainder" must retain their hypothesis status, including any matching derivative assumption, rather than being displayed as a theorem automatically proved about the original function.

## 5.5 Differentiation, exponentiation, and conditioning

Take  $R(x) = x^2 \sin(x^{-3})$ . Then  $R = \mathcal{O}(x^2)$ , but

$$R'(x) = 2x \sin(x^{-3}) - 3x^{-2} \cos(x^{-3}).$$

Along  $x_n = (2\pi n)^{-1/3}$ ,  $R'(x_n)/x_n = -6\pi n$ , so  $R' \notin \mathcal{O}(x)$ . The package's refusal to differentiate a bare magnitude contract is therefore correct.[3]

Similarly, if  $h = H + R$ , then  $e^h = e^H e^R$ . A small *relative* error in  $h$  does not imply a small relative error in  $e^h$  when  $h$  diverges. One needs an appropriate absolute control on  $R$ , or an exact retained inner expression. The source-chart reconstruction and exponentiation paths already encode this distinction; future generalized composition should use it as an invariant.[3, 8]

## 6 Findings registry and priorities

Severity is relative to this library’s purpose. A high-priority finding can be a resource-exhaustion or contract-preservation defect without being a wrong coefficient. “Pending native” means exactly that; source inspection is not presented as an observed Wolfram session.

| ID  | Finding                                                                                     | Priority        | Evidence and disposition                                                 |
|-----|---------------------------------------------------------------------------------------------|-----------------|--------------------------------------------------------------------------|
| A01 | Eager dense classical-series export can overwhelm a sparse computation                      | High            | S, D, E, U. Contain immediately; make conversion lazy.                   |
| A02 | Classical-series export loses logarithmic remainder metadata                                | High            | S, D, E, U. Reject or explicitly describe lossy conversion.              |
| A03 | Inherited observable power silently shadows an explicit option                              | Medium          | S, D, U. Decide and enforce the public precedence contract.              |
| A04 | A fixed replay margin can fail to achieve requested refinement                              | Medium          | S, D, E, U. Propagate precision demands through valuations.              |
| A05 | Coefficient normalization can invoke uncontrolled integer factorization                     | Medium          | S, P, U. Budget and defer nonessential normalization.                    |
| A06 | One resource option measures several incompatible kinds of work                             | Medium          | S, P. Separate budgets and report the exhausted dimension.               |
| A07 | Checked-in CI does not gate native mathematical correctness                                 | High            | S, R, P. Add a pinned, native release attestation.                       |
| A08 | The legacy full-suite runner can accept an empty suite                                      | Medium          | S, P, U. Reuse the stronger focused-runner infrastructure.               |
| A09 | Cutoff and term-goal meanings change with the selected scale                                | Medium          | S, P. Expose a typed request and an execution plan.                      |
| A10 | Convenient numeric evaluation does not check domain or error                                | Medium          | S, P. Add a distinct checked-evaluation interface.                       |
| A11 | Result schemas, capabilities, and saved-head migration are implicit                         | Medium          | S, R, P. Version data and provide capability queries.                    |
| A12 | Native conventions are only partially accepted at the boundary                              | Low–medium      | S, P. Normalize direction aliases and document order differences.        |
| A13 | Certificate coverage and the <code>Automatic</code> interval default need clearer discovery | Low–medium      | S, P. Improve required-argument diagnostics and capability descriptions. |
| A14 | Paclet documentation and license metadata need distribution polish                          | Low             | S, P. Add native help integration and align license identifiers.         |
| A15 | Comparator fallback has a proof-obligation gap                                              | Audit candidate | S, P, U. No wrong public result established; strengthen the invariant.   |
| A16 | Repeated symbolic expansion and linear-power paths deserve profiling                        | Medium          | S, P, U. Optimize only after contract-matched measurements.              |

There is deliberately no entry claiming that the library cannot handle fractional powers, logarithmic inverses, special functions, flat sectors, or automatic series arithmetic. Those capabilities are

already present. There is also no claim that all built-in Wolfram asymptotic results carry a rigorous numeric error bound. Comparisons must distinguish representation, formal expansion, asymptotic theorem, and numerical certification.

## 7 Detailed defects and reproducible contract examples

### 7.1 A01: A sparse answer triggers a dense allocation

**Classification.** High-priority resource defect; S, D, E; native reproduction U.

**Location.** In the core module, `makeForwardObject` calls `makeSeriesData` while constructing every ordinary forward result. `makeInverseSeriesData` performs the analogous conversion for eligible inverse results, including results reconstructed by refinement. Both form a common rational-exponent denominator and then allocate a coefficient list. The essential operation is:

```
den = LCM @@ (Denominator /@ Append[exps, remData[[1]]]);
nmax = remData[[1]] den;
coeffs = Table[0, {nmax - nmin}];
```

No dense-span budget is checked at this point. [2, 5]

**A minimal family.** Let  $q \geq 2$  be an integer and consider

```
AsymptoticExpansion[1 + x^(1/q) + x, {x, 0, 1},
  "MaxTerms" -> 10]
```

The exclusive cutoff retains just 1 and  $x^{1/q}$ ; the first omitted power is 1. Thus the exported lattice has denominator  $q$ , lower index zero, and upper index  $q$ . The requested coefficient-list length is

$$n_{\max} - n_{\min} = q.$$

For  $q = 10^9$ , this is a billion slots, despite only two retained sparse blocks. The input is exact and compact, and its mathematically correct sparse answer is trivial. This observation does not depend on estimates of Wolfram's per-slot memory layout.

The independent checker computes these spans without allocating the lists. The supplied native regression uses the much smaller denominator  $10^6$  and wraps evaluation in time and memory limits. The billion-slot case is *not* executed, and no empirical memory number is claimed.

**Why this is more than a slow optional conversion.** The conversion is eager: a user who never requests `s["SeriesData"]` can encounter its cost. Moreover, `"MaxTerms"` bounds several sparse operations but not this expansion of the representation. A resource guard in a preceding sparse product does not protect the constructor from a later dense export.

**Recommended correction.** Preserve the sparse answer and make classical conversion an explicit, lazy operation. Compute the common lattice and its span first; refuse conversion when the representation budget is exceeded. Return an informative `Missing` or `Failure` from the export request, not from the otherwise successful sparse construction. Cache a successful conversion only on explicit demand, with a versioned cache policy. The plan should report retained blocks, dense slots, denominator, and whether an error contract would be lost.

The supplied patch generator adds a conservative 200,000-slot guard to both exporters. This is an immediate containment proposal, not a substitute for a configurable lazy-export design. It is deliberately limited to the pinned source and does not modify the repository in this review.

**Acceptance criteria.** Construction of the sparse family must remain proportional to its sparse representation, subject to exponent arithmetic; an optional conversion above its budget must return

a structured reason; existing small classical exports must remain unchanged; inverse construction and refinement must obey the same rule. Tests must run against both modular and regenerated standalone entry points.

## 7.2 A02: Logarithmic remainder information disappears on export

**Classification.** High-priority interoperability-contract defect; S, D, E; native reproduction U.

The rich remainder descriptor contains both a power and a logarithmic degree. The two exporters use the power but not `remData[[2]]`. Therefore they cannot preserve that two-dimensional magnitude contract in their current return value. [2]

Consider the finite source expression and cutoff

```
s = AsymptoticExpansion[1 + x Log[x], {x, 0, 1}];
{s["RemainderPower"], s["RemainderLogDegree"]}
(* Expected rich metadata: {1, 1}. Native test supplied. *)
```

The discarded term is  $x \log x$ , so the rich object's natural bound is  $\mathcal{O}(x(1 + |\log x|))$ . The exporter constructs the same ordinary coefficient list and power frontier that it would construct after discarding  $x$  from  $1 + x$ . The missing distinction is mathematically material:

$$\frac{|x \log x|}{x} = |\log x| \longrightarrow \infty, \quad x \downarrow 0.$$

Thus an algebraic magnitude bound  $\mathcal{O}(x)$  cannot replace the richer one.

**Important qualification.** This is not an accusation that native `SeriesData` is intrinsically incorrect for logarithmic expansions. Its formal series-order conventions and the package's explicit magnitude contract are different interfaces. The defect is that a conversion advertised as a useful property silently omits a piece of the package's contract, and a downstream consumer cannot reconstruct it from the exported object alone. Unlike `Normal`, the export does not make the loss explicit.

Three policies are defensible. A conservative export returns `Missing["LogarithmicRemainder"]` whenever the logarithmic degree cannot be represented faithfully. A richer export can return both the classical object and a remainder descriptor with an explicit compatibility statement. A third, deliberately weaker, export can replace the power by  $\rho - \varepsilon$  for an explicitly chosen  $\varepsilon > 0$ , since

$$u^\rho(1 + |\log u|)^d = \mathcal{O}(u^{\rho-\varepsilon}).$$

The third choice must recheck the retained support: the weakened frontier cannot be inserted blindly below terms that remain in the classical list.

The supplied containment patch chooses the first policy. Tests should cover zero and nonzero logarithmic degrees, empty retained lists, inverse exports, and results whose coefficients themselves contain logarithms. A coefficient containing a logarithm is not by itself a reason to reject a formal series; the specific concern is the *remainder contract*.

## 7.3 A03: Explicit observable powers are silently shadowed

**Classification.** Medium-priority option-contract ambiguity; S, D; native reproduction U.

`InverseExpansionCoefficient` accepts either a normalized model or an inverse object. Its model overload reads the public "Power" option. Its object overload delegates as follows:

```
InverseExpansionCoefficient[a["Model"], k,
  "Power" -> inheritedInternalPower, opts]
```



An explicit caller option is placed after an already supplied value. Wolfram’s option-precedence behavior makes that inherited value effective. [2, 33]

For  $f(x) = x + x^2$ , let  $g = f^{-1}$  near zero. Direct algebra gives

$$g(y) = y - y^2 + 2y^3 + \cdots, \quad g(y)^2 = y^2 - 2y^3 + \cdots.$$

The first correction coefficient is  $-1$  for power one and  $-2$  for power two. Consequently, an object query that accepts **Power**  $\rightarrow 2$  but retains the original power-one computation is observably different from the corresponding model query. The independent checker verifies the algebra; the native suite supplies both queries.

There are two reasonable API contracts. An object coefficient query can mean “a coefficient of exactly this stored observable”, in which case a conflicting power option should be rejected. Alternatively, it can mean “a coefficient from this stored forward model for the requested observable”, in which case an explicit power should override inheritance. Neither contract justifies silently accepting an ineffective option.

The included helper `InverseCoefficientForPower` implements the second policy for ordinary power-log inverse objects. It first resolves the *external* requested observable power and only then converts to the internal exponent required at a source infinity. Simply moving the option to the front is not an adequate general repair: the sign conversion must still be applied to the explicit value. The helper also rejects unsupported kinds instead of indexing a model they do not carry.

**Acceptance criteria.** Test inherited and explicit powers at finite and infinite source endpoints, negative branches, invalid powers, model queries, and nonordinary result kinds. Document whether a coefficient query changes the observable or only inspects an existing object. Any chosen policy should be consistent across the general and Fourier-specific coefficient interfaces.

## 7.4 A04: Refinement can return a valid result below its requested goal

**Classification.** Medium-priority goal-satisfaction defect; S, D, E; native reproduction U.

The generic operation-replay branch of `SeriesRefine` refines each operand using a guard margin based on the requested cutoff and the old operand precision. It does not generally solve the inverse precision problem for the operation. The source uses a margin of the form

$$h + 2 + |\min(0, \rho_{\text{old}})|.$$

The arithmetic afterward still transports the available remainders. This is good for correctness of the returned bound, but does not ensure that the requested goal has been achieved. [3]

A concrete source trace is:

```
s = AsymptoticExpansion[Exp[x], {x, 0, 2}];
t = AsymptoticExpansion[x^-100, {x, 0, 1}];
p = SeriesMultiply[s, t];
r = SeriesRefine[p, 5];
```

The exact pole operand requires no unknown-tail improvement. For the exponential operand, the generic replay asks for cutoff  $5 + 2 = 7$ . Multiplying its  $\mathcal{O}(x^7)$  tail by  $x^{-100}$  gives only  $\mathcal{O}(x^{-93})$ , whereas the requested product goal is  $\mathcal{O}(x^5)$ . The exponential operand actually needs precision 105.

This is *not* a claim that the multiplication invents coefficients or returns an invalid bound. Its precision calculus clips to what is known. The problem is a missing guarantee or explicit failure at the refinement request boundary. An object can carry the requested goal in its history without achieving that goal in its returned error grade.



The right repair is backward precision planning. For a product with leading valuations  $\alpha = \nu(A)$  and  $\beta = \nu(B)$ , the first necessary power requirements for output precision  $H$  are

$$\rho_A \geq H - \beta, \quad \rho_B \geq H - \alpha.$$

One must also check the error–error product and logarithmic envelope degrees. If the available leading valuation is unknown because of cancellation, obtain a leading-term certificate or refine it first; do not silently assume zero valuation. The supplied Python check and Wolfram helper verify the simple 105 demand, but do not claim to implement the complete planner.

**Acceptance criteria.** A refinement request should return either a result whose reported achieved grade meets the requested grade, or an explicit failure carrying the best available result and the reason for shortfall. Tests should include large poles, reciprocals near cancellation, powers of a vanishing series, compositions with large valuation multipliers, and operations mixing exact carriers with asymptotic brackets.

## 7.5 A05: Normalization hides an integer-factorization problem

**Classification.** Medium-priority performance and predictability risk; S, P; native timing U.

The core’s `logCanon` expands logarithms of positive integer and rational constants using `FactorInteger`. It is called within coefficient and polynomial canonicalization. Thus an input containing an otherwise harmless constant such as  $\log N$  can introduce the cost of factoring  $N$  before the requested asymptotic computation needs that factorization. [2]

For a family  $N = pq$  with large distinct primes, the expression  $\log N$  has a compact exact representation. Factoring it is unnecessary for returning an expansion such as  $1 + (\log N)x + \mathcal{O}(x^2)$ . The concern is the coupling of routine normalization to an expensive unrelated algebraic task, not a measured assertion that every large integer is difficult. No expensive factorization fixture was run in this audit.

A safer normalizer keeps  $\log N$  atomic by default, performs cheap rational and structural simplifications first, and invokes factorization only under an explicit effort budget or for sufficiently small constants. A request-local cache can avoid repeating successful work. A cache of failed or timed-out canonicalizations must include the assumptions and effort policy so that a later stronger request is not suppressed accidentally.

This change creates a genuine tradeoff: aggressive normalization can prove that two independently written coefficients cancel, and therefore can improve the selected frontier. The solution is not to replace exact equality with numerical equality. It is to make unresolved equality and expensive normalization visible, preserve a conservative remainder when a cancellation is unproved, and allow users to opt into stronger simplification.

## 8 Audit of mathematical and implementation invariants

### 8.1 Sparse ordering, resonances, and the comparator candidate

The sparse engine merges equal weights, combines coefficient polynomials, and sorts the result. Algebraic canonicalization is important: for example,  $\sqrt{8}$  and  $2\sqrt{2}$  must not create different layers. Multi-index resonances are equally important. If gaps are 1 and 2, the multi-indices  $(2, 0)$  and  $(0, 1)$  contribute to the same weight and must be combined before the layer is counted, declared zero, or omitted.

The implementation already contains complete-layer and complete-boundary logic, including retained refinement state. These are substantive features, not proposed additions. Testing should

stress their invariance under permutation of forward blocks, algebraically equivalent weights, and cancellation of an entire first omitted layer. [2, 5, 6]

A narrowly scoped candidate deserves attention in the exact comparator. After several earlier equality and sign tests, the fallback simplifies  $d < 0$  and returns a positive comparison when that predicate simplifies to **False**. Logically,  $\neg(d < 0)$  proves only  $d \geq 0$ , not  $d > 0$ . The earlier equality tests may already settle this for the intended exact numeric inputs; this review has not established a public counterexample. The finding is therefore A15, an invariant-hardening opportunity, not a confirmed incorrect exponent order. [2]

The robust contract is a three-way proof procedure: establish  $d = 0$ , establish  $d < 0$ , or establish  $d > 0$ . Otherwise return an undecidable-order failure. For symbolic weights under assumptions, keep that procedure separate from a comparator whose precondition is a fixed exact real number. Never repair the gap using a fixed-precision numerical sign test near zero.

## 8.2 Native-series ingestion is an analytic boundary

The native special-function adapter is not merely a call to `Normal[Series[...]]`. It recognizes candidate heads, obtains native series information, and then applies checks on the supported representation, realness, coefficients, and error envelopes. A list of recognized heads is not a promise of all parameter values and endpoints for those functions. The native adapter also contains a conservative weakening of a power frontier to absorb logarithmic growth in a tail. [7]

That boundary needs especially adversarial tests. A native expression can contain a nested or multiplied `SeriesData`, a retained exponential carrier, a branch-dependent coefficient, or a coefficient whose apparent zero is conditional on parameters. Every accepted conversion should record which part of the native result justified the retained blocks and which part justified the error envelope. A fallback must not turn a missing native remainder into a guessed exact finite function.

Useful cases include expansions with their first several coefficients zero, logarithmic singularities at a finite point, parameter values that cancel the generic leading coefficient, and special-function identities that are real only on a restricted source approach. The review does not establish a wrong native-ingress result for such a case; these are high-value regression candidates. A deliberately unsupported case returning a precise failure is preferable to an unjustified broad acceptance claim.

## 8.3 Fourier coefficients and oscillatory zeros

The Fourier engine stores finite Fourier-polynomial modes in the logarithmic variable and validates real conjugacy relations. It refuses an oscillatory leading block without an eventual nonzero sign. Its remainder explanation uses coefficient norms and polynomial envelopes, not the value of one oscillatory factor at a numerical sample point. This is the appropriate principle: a vanishing  $\sin(\omega \log u)$  is not evidence that an entire unknown remainder vanishes. [6]

Future Fourier work should preserve exact frequencies through products and compositions. Frequency truncation needs its own error bound; a `"MaxFrequencies"` resource failure must not be replaced by dropping small-looking modes. Test zero-frequency cancellation, conjugate pairs, repeated frequencies, resonant source weights, and agreement of the general residual dispatcher with `FourierInverseResidual` on supported objects. The dedicated Fourier coefficient accessor already exists and should be advertised by a capability query rather than obscured behind a generic wrong-kind failure.

## 8.4 Finite flat sectors are not a universal transseries field

The repository has explicit finite flat-sector construction and operations. That is a significant extension beyond an ordinary power–log jet, but it should remain described as its implemented finite scale calculus. A general well-based logarithmic–exponential transseries field requires closure, comparison, composition, and support-management guarantees beyond a set of family-specific reductions. Those guarantees cannot be inferred from the presence of a function named for a transseries-related operation. [1, 4]

A good invariant is that omitted sectors are represented by a genuine sector envelope, not by an algebraic power that cannot dominate the omitted exponential on the selected approach. Cancellation of exact sectors should not erase an uncertain empty jet. The current arithmetic code has distinct flat and composite routes; changes should preserve that distinction instead of forcing all results into an inappropriate single power cutoff.

## 8.5 What the exact interval certificate proves

The certificate evaluator encloses elementary expressions with exact rational endpoints and directed dyadic rounding. Exponential and logarithmic enclosures use explicit series tails; products, reciprocals, and permitted powers preserve interval inclusion. The code checks the selected source side and the retained source-domain condition on the entire closed interval. It then requires the derivative to be separated from zero. [9]

The underlying argument can be stated independently. Suppose  $f$  is continuous on an interval  $I$  and differentiable in its interior, with  $|f'| \geq \mu > 0$  and a constant derivative sign. For a center  $c \in I$ , an exact residual enclosure implies  $|f(c) - y| \leq \varepsilon$ . If  $[c - \varepsilon/\mu, c + \varepsilon/\mu] \subseteq I$ , monotonicity and the mean-value theorem establish a root between those endpoints, and strict monotonicity gives uniqueness. Alternatively, certified endpoint signs can establish existence. A signed derivative enclosure can then sharpen the root interval. These are the two existence routes visible in the code.

For example, for  $f(x) = x + x^2$ ,  $y = 1/10$ ,  $c = 1/10$ , and  $I = [0, 1/5]$ , the residual is  $1/100$  and  $f' \geq 1$ . The bracket  $[9/100, 11/100]$  lies in  $I$ . The independent check verifies these rational inequalities; it does not execute the package certificate function.

Crucially, the returned certificate says that it does not certify the whole family of functions compatible with a declared `InputRemainder`. Nor does uniqueness on one verification interval establish uniqueness of a global inverse branch. These limitations are already made explicit by the implementation and should remain prominent in the user interface. [9]

An independent certificate checker would improve assurance further. The current evaluator is mathematical software, not a machine-checked proof of its own implementation. A compact transcript containing the exact interval operations, tail inequalities, domain checks, and final containment test could be replayed by a smaller checker. Such a checker would validate the particular certificate, not establish correctness of every heuristic used to choose its initial center.

## 8.6 Distinguishing three kinds of residual evidence

A formal jet residual verifies algebraic cancellation below a specified grade in a stored finite model. A numerical residual or a numerical reference-root comparison supplies finite-precision evidence at selected points. An interval certificate proves a local statement for an explicit function with the enclosures it actually supports. The library exposes all three kinds, and their names, return fields, and documentation should keep them separate. [2, 9, 10]

A zero residual in a finite Stirling model does not remove the original Stirling tail. A tiny residual

need not imply a small inverse error when  $f'$  is small. Agreement at a handful of points cannot resolve a branch ambiguity. These are reasons to improve provenance presentation, not reasons to discard the useful formal and numerical tools.

## 9 Performance analysis and resource design

### 9.1 The relevant complexity parameters

A benchmark indexed only by the requested number of terms misses most of the important costs. At least six quantities matter: retained support size, number of candidate products, logarithmic polynomial degree, arithmetic complexity of exponents, number of independent gaps, and complexity of the symbolic coefficient field. Classical export adds a seventh, the dense lattice span.

For  $m$  positive gaps, the multi-index set

$$\{k \in \mathbb{N}^m : k \cdot \delta < H\}$$

has a leading volume estimate  $H^m/(m! \prod_j \delta_j)$  as  $H$  grows with fixed positive gaps. This is a lattice-point estimate, not a package timing prediction. Small gaps or many independent gaps can create many candidate indices even when resonances merge them into comparatively few displayed powers. An implementation that limits only returned blocks does not bound this work.

For jets of sizes  $n$  and  $m$ , an untruncated product has  $nm$  candidate pairs before merging. The current `jetMul` already uses sorted weights and a monotone scan to skip products beyond the cutoff and to check a pair budget before multiplying the coefficient polynomials. Similar pruning was added to Fourier products. These optimizations should be retained and regression-tested; proposing a Cartesian product followed by filtering would move backward. [2, 6, 13]

### 9.2 A06: one option is not one budget

The public "`MaxTerms`" value is used in different places for retained or candidate terms, index regions, product counts, polynomial expression size, and model orders. Other paths use a frequency cap, hard-coded simplification timeouts, or chart-recursion limits. A user who raises "`MaxTerms`" cannot predict which resource has actually been granted. [2, 7, 11, 8, 5]

A structured budget should distinguish at least `MaxBlocks`, `MaxCandidateProducts`, `MaxIndices`, `MaxCoefficientLeaves`, `MaxDenseCoefficients`, `MaxFrequencies`, and a total time or work budget. The existing option can remain a backwards-compatible shorthand during migration. Every resource failure should name the dimension, the requested or observed amount where available, and whether any validated partial result exists.

Local symbolic subcalls should consume the caller's remaining budget rather than each receiving a fresh hidden allowance. A timeout should mean "proof or computation not completed under this budget", not a negative mathematical conclusion. In particular, a timed-out sign proof cannot be used as evidence of nonpositivity, and a failed simplification cannot be used as evidence that two expressions differ.

### 9.3 A16: candidates for improvement, not measured speedups

The native adapter includes a repeated-multiplication path for positive integer powers, while the ordinary and Fourier algebras have more efficient power-building structures. Binary powering with early truncation is a reasonable candidate for the native path, but its envelope normalization cost

must also be measured. Intermediate expression growth can dominate the number of multiplications. [7, 2, 6]

The Gamma inverse coefficient builder repeatedly constructs and expands a phase through successively increasing orders. This is easy to understand and valuable as a reference implementation. A retained coefficient cache, incremental truncated polynomial arithmetic, or a Newton-style update could reduce repeated work. However, the main recommendation is to benchmark representative orders before replacing a transparent recurrence. The current code’s `LeafCount` guard is useful but does not necessarily bound the cost of a symbolic expansion that occurs before the guard. [11]

Coefficient normalization should be staged. Cheap exact arithmetic, structural zero checks, and algebraic canonicalization should precede expensive global simplification or factorization. Request-local memoization can reuse exponent comparisons and coefficient normal forms. Cache keys must include assumptions and the relevant normalization policy. Do not introduce an unbounded global cache of expression trees or silently reuse facts established under incompatible assumptions.

For ordinary inverse refinement, model signatures, retained coefficient prefixes, and residual checks of recovered state already exist. This is a stronger starting point than replaying every request from zero. An automatic method selector can build on `Lagrange`, `Newton`, and grouped methods already present: estimate index-region size, degree growth, and expected resonance before selecting a route. The selected method and any fallback should be recorded in a plan. [5, 2]

## 9.4 Benchmark protocol and the supplied harness

The included `CompareBuiltins.wls` measures selected package and native operations, subject to per-evaluation time and memory limits. It records actual outputs, messages, result heads, object size, finite-expression size, and time for extracting the normal expression. Its “operation-cold” measurement clears the symbolic system cache; it is explicitly *not* a fresh-process cold start. Warm observations are recorded separately. No numbers from that harness are present because it has not been run here.

A credible future comparison should separate loading, construction, normalization, presentation, classical export, and refinement. The package object contains metadata and sometimes reusable state, whereas a native result may be just a formula. Comparing only their total byte counts or construction times can be misleading. Compare normal expressions and achieved mathematical information as well as complete objects.

The two systems’ order arguments are not automatically equivalent. An exclusive package cutoff of four does not necessarily match a native “order four” request. Select a common retained prefix or prove residual agreement at a common grade before computing a speed ratio. For implicit native solutions, branch selection and the number of returned branches are part of that matching task.

Suggested workloads are small Taylor/Puiseux baselines; irrational gaps; resonant logarithmic inverses; high-denominator sparse powers; large-pole refinement; cancellation-heavy arithmetic; Fourier frequency growth; and Gamma/Barnes exact-core inverses. Use exact data, fixed assumptions, pinned kernel and package versions, multiple repetitions, and recorded failures. A timeout is a censored observation, not an arbitrary large successful runtime. Repository-reported operation improvements should remain labeled as local package-to-package measurements, not generalized into built-in comparisons. [13]

## 10 API and user-experience redesign

### 10.1 A09: make the order request a typed mathematical object

The same-looking cutoff argument has different meanings in different families. That flexibility is mathematically natural, but it is difficult to discover from a call that selects its scale automatically. [2, 10, 11, 1]

| Representation                        | Order coordinate                               | Important qualification                                                                         |
|---------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Ordinary power-log                    | Positive local source or target power          | Cutoff is exclusive; coefficients are complete logarithmic blocks.                              |
| Lambert/logarithmic reduction         | Inverse-logarithmic variable in a unit bracket | The extracted prefactor is not counted as an ordinary power of the original target.             |
| Gamma/Barnes ordered inverse          | Reciprocal exact core                          | Each core-power block retains its polynomial in an inverse logarithm.                           |
| Legacy explicit Gamma special adapter | Exact-core perturbation marker                 | The integer argument denotes an inclusive marker depth, not the same ordered core-power cutoff. |
| Finite flat sectors                   | Sector support and within-sector grades        | A single ordinary power cutoff cannot describe all omitted sectors.                             |
| Composite error envelope              | Recorded compatible approach                   | There may be no single scalar cutoff; operand truncation must be explicit.                      |

Retain concise calls for standard use, but expose the resolved request:

```
(* Proposed interface, not an implemented repository feature. *)
SeriesPlan[f, {x, x0},
  <|"Goal" -> "PowerCutoff", "Value" -> h,
    "Coordinate" -> Automatic, "Boundary" -> "Exclusive"|>]
```

A plan should explain the source approach, target approach, selected scale, exact carrier, meaning of a block, transported input ceiling, expected backend, and resource limits. A result should contain both `RequestedPrecision` and `AchievedPrecision`, together with an explicit `GoalSatisfied` value. A request that cannot be meaningfully compared to the selected scale should fail before performing substantial coefficient work.

The project already accepts structured refinement requests for additional ordinary inverse blocks and for certified numerical tolerances. The additional-block route reports whether the goal was reached, preserves a best expansion on resource failure, and distinguishes exact termination. The numerical route explicitly says that its certificate does not improve the symbolic series remainder. The proposed general precision interface should extend this existing design to numeric-cutoff operation replay, not replace it or claim to introduce structured requests for the first time. [19]

A term goal also needs a cancellation policy. “First  $n$  nonzero complete blocks” is not the same as “first  $n$  multi-indices” or “first  $n$  monomials after expanding every logarithmic polynomial”. The package’s complete-block convention is a good default; display it alongside the returned count. For exact finite answers, returning fewer blocks should report exact termination rather than look like an incomplete computation.



## 10.2 A10: keep approximate evaluation distinct from checked evaluation

The object accessor `s[value]` substitutes into the finite expression. It does not call a domain prover, solve the original equation, or return an error enclosure. This is documented behavior, so it is not an implementation bug. Nevertheless, the notebook presentation can make it look more certified than it is, especially when the head and metadata are hidden. [2]

A separate proposed `SeriesEvaluate` interface should distinguish a finite approximation, a numerical diagnostic, and a certified enclosure. An approximation result should report whether the point satisfies known domain predicates, whether that check is proved or merely numerical, and whether an error bound is quantitative or only asymptotic. A symbolic Big-O class without a constant and validity radius cannot produce certified error bars at a chosen point.

The inexpensive accessor can remain, but a compact summary panel should make its status clear: scale, branch, retained blocks, known precision, certificate availability, and whether the displayed formula has dropped anything. Do not place an unconditional green “certified” badge on a series whose only evidence is a formal zero residual.

## 10.3 A11: version the result schema and expose capabilities

Different result kinds carry different fields. An ordinary inverse has a `Model`; a Fourier inverse has frequency-specific normalized data; a special-function object can retain a coordinate series and an exact carrier. Operations currently dispatch by combinations of kind, scale, and field presence. This works, but makes maintenance and error messages fragile as new families are added. [2, 3, 6, 10]

Introduce a small validated schema with stable mandatory fields and kind-specific extensions. A capability query should answer whether an object supports ordered truncation, additional blocks, formal residuals, coefficient queries, classical export, differentiation, and numeric certification. Each answer should carry a reason and, where appropriate, the additional required hypotheses. Unsupported operations should return a precise kind-aware failure, not a generic invalid-argument message arising from delegation to a missing model.

The 1.8.0 validation record states that `GeneralizedSeries` replaced the previous public head and that the old head is not exported as an alias. That intentionally affects explicit patterns and saved input. Provide a migration function for supported saved structures and document which older representations cannot be safely upgraded. Silent rewriting of arbitrary associations would be dangerous: changed field meanings require validation, not just head replacement. [13]

Version computation state separately from display data. Serialized state should include the package/schema version, model signature, assumptions, branch, coordinate definitions, normalization policy, and source precision. The current signature and compatibility checks provide a useful foundation. User editing of metadata should invalidate cached state unless a validator reestablishes the required invariants. [5]

## 10.4 A12: normalize native conventions without hiding differences

The ordinary local-coordinate function accepts `Automatic`, `"FromAbove"`, and `"FromBelow"`, rejecting other direction forms. Numeric Wolfram direction aliases are not accepted by that path. This is an interoperability omission, not a discovered wrong-branch answer. A central normalization layer can map supported aliases to the package’s real approach representation before dispatch, while continuing to reject complex directions outside its contract. [2, 22]

Assumption handling should also be explicit. Several public option lists use `Assumptions -> True`, whereas native functions commonly use `$Assumptions`. Some internal simplification calls

may still interact with the global environment; this review does not assert exact behavior under every **Assuming** wrapper without native execution. Add tests that compare explicit assumptions, a dynamic assumptions block, and a retained object’s later refinement. Choose a policy and retain the effective assumptions with the result. [2, 21]

Use consistent failure categories for malformed syntax, unsupported scale, unproved branch condition, insufficient input precision, exhausted budget, and numerical diagnostic failure. These conditions suggest different remedies. For example, increasing the number of terms cannot repair a missing derivative hypothesis, and increasing a time budget cannot justify a complex branch in a real-only scale.

## 10.5 A13–A14: certificate discovery and distribution polish

`InverseCertificate` has an `"Interval" -> Automatic` option default, but its current implementation requires an explicitly supplied ordered rational interval. The usage text does show that interval, so this is a low-priority sentinel-design issue, not a hidden undocumented mathematical requirement. A missing-interval diagnostic should say what is required. A future automatic bracket search must preserve the selected branch and prove every successful bracket; failure to find one is not proof that no inverse exists. [9, 2]

The paclet manifest declares only a kernel extension. The repository’s Markdown/HTML guide is substantial, and the validation record reports that all 37 public symbols are documented. Native help-system integration, searchable symbol pages, executable examples, and a guide organized by scale and supported operation would improve discoverability without replacing those materials. [12, 13]

The root license text is titled “MIT No Attribution License”, while the paclet metadata says “MIT”. Align these literal metadata identifiers so packaging tools and users see a consistent declaration. This is a metadata-consistency observation, not a legal opinion about licensing obligations. [18, 12]

The project already documents problems with direct cold HTTP loading of a large standalone file and recommends downloading completely before `Get`. Preserve that mitigation. Release instructions should offer a commit-pinned file or paclet with a published checksum and an offline path, rather than encouraging evaluation of a moving `main` URL. The standalone builder’s dependency checks and embedded module hashes are valuable and should remain part of the release process. [15]

## 11 Testing, CI, and release evidence

### 11.1 A07: packaging CI is not mathematical CI

The inspected `standalone.yml` workflow checks that the generated standalone source is fresh and runs the Python builder tests. These checks are useful: stale generated code is a real distribution defect. They do not execute the Wolfram package or compare its mathematical outputs. The latest focused native validation is a separate, repository-reported activity, not a native gate in this workflow. [16, 13]

Add a native job on a properly licensed runner, or a release procedure with equivalent auditable evidence when such a runner is not available for every pull request. The release gate should load a clean pinned checkout, regenerate and compare the standalone file, run both focused and complete native suites, exercise the modular and standalone public entry points, and retain the actual kernel version, platform, source hashes, selected tests, counts, messages, and failures. Unsupported or skipped tests must remain visible.



A useful sequence is fast structural and builder checks on every relevant change, a focused native matrix selected from touched subsystems, and a full clean-process native suite before release. A focused pass must not be relabeled as a full pass merely because all tests selected by that runner succeeded. The current validation documentation is commendably explicit about this scope; automation should preserve the same honesty.

## 11.2 A08: bring the full-suite runner up to the focused-runner standard

The inspected `Tests/RunTests.wl` enumerates `*.wlt` files and exits according to the failed-test count. It does not first reject an empty file list or require a positive executed-test count. Therefore an accidental empty selection is not explicitly prevented from appearing successful by this runner. It also lacks the source-hash attestation described for the newer focused-runner infrastructure. [17, 13]

The repository already has a stronger focused runner that checks loading, rejects empty suites, records sources before and after execution, and propagates export failures. Reuse or extract that infrastructure instead of creating yet another independent runner. The finding does not apply to all upstream test scripts. Add a fixture that deliberately points to an empty test directory and requires a failing process status; also test missing output directories and report-export failures.

## 11.3 The highest-value new test classes

A coefficient oracle alone is insufficient for a library whose main value is metadata. Every representative fixture should check the expression, selected approach, coordinate, remainder scale, exactness status, achieved precision, and provenance appropriate to its kind. A correct displayed formula with a false or lost contract is a failing result.

Property-based tests should randomize small exact finite models and verify that the inverse residual vanishes below the requested internal grade. Compare independent Lagrange and Newton routes on their overlap, then compare a classical specialization with native reversion. Random generation must preserve admissibility; an expected failure on a genuinely unsupported scale is not a failed mathematical algorithm.

Metamorphic tests are especially useful here. Permuting or regrouping input terms, replacing an algebraic exponent by an equal expression, applying a known affine coordinate change, and refining in one step versus several steps should preserve the appropriate contract. Truncation must never strengthen an unknown remainder without new evidence. A refinement cache with changed assumptions or a changed branch must be rejected. A generated standalone package must agree with the canonical modular entry point.

Negative tests should cover invalid numeric directions, approximate input, nonreal branches, ambiguous leading signs, missing derivative bounds, input-precision ceilings, dense representation budgets, frequency budgets, and timed-out proof attempts. Check failure tags and the absence of noisy internal messages, not only whether the result matches `_Failure`.

Numerical tests should use exact starting parameters, adequate working precision, and multiple approach points, ideally with a normalized residual or error divided by the claimed scale. Near zeros of an oscillatory factor, use the established envelope rather than that factor itself. A failed `FindRoot` reference computation should be recorded separately from a failed asymptotic approximation.

## 11.4 What was actually run for this report

| Evidence                                       | Result                            | Scope                                                                                       |
|------------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------|
| Independent mathematical checker               | 16 passed; 0 failed               | Python/SymPy identities, span calculations, and precision examples.                         |
| Python audit-tool unit tests                   | 7 passed; 0 failed                | Patch transformation, source-shape rejection, exact span calculation, and Git-blob hashing. |
| Package native regression tests in this review | Not executed                      | Native connector failed; no local Wolfram kernel.                                           |
| Proposed Wolfram helpers and patched exporters | Not executed natively             | Provided as reviewable proposals, not validated replacements.                               |
| Built-in comparison benchmarks in this review  | Not executed                      | Harness provided; no measured speedup claimed.                                              |
| Latest upstream 1.8.0 focused record           | 305 passed; 0 failed, as reported | Selected 18 files; full suite explicitly not run at that milestone.                         |

The included evidence files preserve these categories. In particular, there is no fabricated `native-audit-results.json` or built-in benchmark result. Those files are created only if the supplied native runners are actually executed.

## 12 Proposed changes and use of the accompanying artifacts

### 12.1 A deliberately small containment patch

`patches/make_export_patch.py` verifies the pinned core's Git blob identifier before proposing a modification. Its default behavior prints a unified diff and does not write the repository. An explicit `-in-place` request creates a backup and atomically replaces only the canonical core file. Source drift, a missing definition, or an already applied patch is rejected. Line endings are normalized for the source-hash comparison.

The patch does two things in each classical exporter: it returns an explicit missing representation for a nonzero logarithmic remainder degree, and it checks the dense coefficient span before allocating. It does not change coefficient mathematics, implement lazy properties, fix refinement planning, or settle the observable-option policy. The patch is intentionally small enough to review independently of those larger changes.

The exact exporter excerpts used by the patch-tool unit tests are included with provenance and the upstream license. They are *not* a complete package and must not be loaded as one. The whole repository was not copied into this archive. The source-shape tests validate the transformation of those excerpts; they are not native execution of the modified package.

A typical workflow, using a separate local checkout at the pinned revision, is:

```
# From the audit archive; replace the path with your pinned checkout.
python patches/make_export_patch.py \
  /path/to/Asymptotic/AsymptoticInverse/Kernel/AsymptoticInverse.wl

# Only after reviewing the diff, request an actual local change:
python patches/make_export_patch.py \
  /path/to/Asymptotic/AsymptoticInverse/Kernel/AsymptoticInverse.wl \
  --in-place
```

```
# From the upstream repository root, rebuild the generated distribution:
python validation/build_standalone.py
python validation/build_standalone.py --check
```

Native regression tests and comparison of modular and standalone loading must follow. No remote repository write or pull request was performed.

## 12.2 Helpers and native test organization

`code/AuditSupport.wl` contains proposed, namespaced helpers. They do not install overrides into the upstream package. `DenseSeriesPlan` preflights a rational lattice and its remainder compatibility without allocating the dense coefficients. `InverseCoefficientForPower` resolves an explicit observable power before source-infinity conversion. `RequiredProductPrecision` states the necessary valuation-based power demands for a product, with its logarithmic and error-product limitations explicitly recorded.

The native suite separates baseline behavior from desired contracts. `Baseline.wlt` tests classical and logarithmic inverses, formal residuals, the rich remainder, model coefficient queries, and a simple explicit-function certificate. `Support.wlt` targets the supplied helpers. `ProposedContracts.wlt` encodes the desired behavior for A01–A04; it is *not* claimed to pass the pinned unmodified package. The A03 test chooses explicit-option precedence, while the article also recognizes the alternative of rejecting a conflicting option.

Set `ASYMPTOTIC_PACKAGE` to an existing local modular core or standalone file, then use the supplied runner. It requires a nonempty suite, records the actual kernel and source file hash, and exports per-test details. Its reference commit is descriptive, not proof that a caller’s entire checkout is clean. A production release attestation should hash all loaded modules.

```
# POSIX shell example; the README also describes the environment settings.
export ASYMPTOTIC_PACKAGE=/path/to/Asymptotic/AsymptoticInverse.wl
wolframscript -file tests/RunAudit.wls

# Desired future contracts (not expected to pass unmodified upstream):
export ASYMPTOTIC_AUDIT_SUITE=Proposed
wolframscript -file tests/RunAudit.wls

# Separate, unexecuted comparison harness:
wolframscript -file benchmarks/CompareBuiltins.wls
```

## 12.3 Independent reproduction and document build

The Python checker can be rerun without a Wolfram kernel. SymPy is its only nonstandard Python dependency. The Python unit tests also load that checker and consequently need SymPy. The article uses standard LaTeX packages and can be compiled without the upstream repository or network access once those packages are installed.

```
python code/independent_checks.py
python -m unittest discover -s tests -p 'test_python_tools.py' -v
cd article
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

These commands reproduce the independent checks and the document, not the native package validation that remains outstanding.

## 13 Development roadmap with acceptance gates

### 13.1 Stage 1: preserve existing information and bound representation costs

First contain A01 and A02. Make dense export bounded, and prevent an export from silently discarding a logarithmic error contract. Select and document the A03 observable-option policy. Add native baseline and regression fixtures before broad refactoring, and ensure the generated standalone source is regenerated from the same canonical modules.

The completion gate is not merely “the new tests pass”. Existing small exports, inverse expressions, remainder fields, and formatting must remain consistent on the native suite. Resource tests must confirm that an optional representation failure does not destroy a valid sparse result. A release record must identify the complete source set actually tested.

### 13.2 Stage 2: make requests and results comparable

Introduce a precision descriptor that identifies the coordinate, exact carrier, power grade, logarithmic grade, sector information where relevant, and hypothesis status. Add requested-versus-achieved reporting. Implement backward demands first for ordinary sums, products, and powers, then for logarithm, exponential, and composition. Keep safe fallback results, but label an unmet goal explicitly.

The acceptance suite should include the large-pole example, exact zero, cancellation that hides a leading valuation, and source remainder ceilings. For every operation, write a precise rule for how input precision implies output precision. New rules must not erase input uncertainty merely because the retained polynomials cancel.

### 13.3 Stage 3: separate planning, algebra, evidence, and presentation

A maintainable architecture can be organized around four interfaces. A planner selects a scale and computes precision demands. A scale algebra implements exact operations and support ordering. An evidence layer records formal residuals, analytic hypotheses, numerical diagnostics, and interval proofs. A presentation layer displays expressions and handles intentionally lossy exports.

This is not a recommendation to rewrite every module at once. Introduce adapters around existing implementations and preserve them as reference backends. Start with ordinary power–log objects, where the invariants are clearest. A scale interface should define support comparison, block normalization, truncation, operation precision, and available evidence. A registry then replaces scattered field-presence tests with explicit capabilities.

The completion gate is agreement with the existing backend on supported cases, clearer failures on unsupported combinations, and no regression in state-signature checks. Only after those invariants are stable should the new interface become the foundation for further scale families.

### 13.4 Stage 4: optimize where measurements identify bottlenecks

Profile canonicalization, index enumeration, coefficient polynomial work, repeated Gamma expansions, and classical export separately. Add bounded request-local caches and improved powering where justified. Select inverse methods from estimated work, with reproducible fallback decisions.

The gate is a contract-matched benchmark suite with raw observations, messages, achieved grades, and both cold-process and warm-operation modes. A smaller expression or faster first call is not a success if it loses a branch condition, a remainder degree, or refinement provenance.

### 13.5 Stage 5: expand analytic scope with explicit proof obligations

Add new adapters only when their admissible domain, forward model, remainder, derivative information, coordinate transformation, and inverse conditioning are specified. Each adapter should provide an unsupported-case boundary and tests at degenerate parameter values. A broad recognized-head list without those contracts is less useful than a smaller reliable domain.

Prioritize extensions that reuse existing exact cores and error transport. Gamma/Barnes interval evidence, additional threshold models, and controlled source-chart composition are natural opportunities. Complex sectors and multivariate systems require a separate architectural decision rather than an accumulation of exceptional dispatch rules.

## 14 Longer-term mathematical opportunities

### 14.1 Certified special-function inverses

The repository already provides Gamma/Barnes and Erfc asymptotic machinery and quantitative forward-model information in explicit adapters. The next step is not “implement these inverses”; it is to connect admissible forward-tail bounds and derivative bounds to a trustworthy original-function inverse enclosure. That may require additional exact constants, special function enclosures, and interval-domain checks beyond the current elementary certificate evaluator. [10, 11, 9]

An adapter contract can return an explicit model  $F_m$ , a bound  $|F - F_m| \leq B_m$  on a specified interval, and derivative information sufficient to separate  $F'$  from zero. The residual bound becomes the sum of the model-evaluation enclosure and the forward-model error bound, rather than treating  $F_m$  as the original function. Any unknown multiplicative constant in a bare Big-O must be resolved before this becomes a quantitative certificate.

### 14.2 Uniformity in parameters and transition models

Current exact parameter assumptions help establish realness and signs, but an expansion valid for each fixed parameter need not be uniform when that parameter approaches a degeneracy. For instance, in

$$f_\varepsilon(x) = \varepsilon x + x^2,$$

the ordinary inverse for fixed  $\varepsilon > 0$  starts with  $y/\varepsilon$ , whereas at  $\varepsilon = 0$  the positive branch is  $\sqrt{y}$ . The crossover occurs when  $y$  is comparable to  $\varepsilon^2$ . No number of additional terms in the fixed-parameter expansion makes it uniform across that transition without changing the scale or retaining a suitable exact core.

A future plan can report parameter-dependent dominant balances and suggest an exact transition core such as  $(-\varepsilon + \sqrt{\varepsilon^2 + 4y})/2$ . Uniformity should be a separate proved property, with a parameter region and scaled target domain. This is a promising direction because it uses the project’s core/perturbation architecture rather than fighting it.

### 14.3 Complex sectors and branch geometry

Extending a real germ to complex sectors requires more than permitting complex constants through input validation. Powers and logarithms need branch data; asymptotic dominance depends on the approach direction; exponentials can change from decaying to growing; and a real interval certificate no longer supplies the relevant existence theorem.

A suitable representation would retain a sector, branch cut or logarithm determination, and admissible comparison relation. Start with a narrow sectorial family whose remainders are already

known, and keep it distinct from the real engine. Stokes transitions and exponentially small terms should be described only to the extent actually implemented. The current finite-sector or real-flat machinery is not by itself a resurgent-analysis system.

#### 14.4 Multivariate implicit inversion

A system version would replace the scalar derivative lower bound with Jacobian invertibility and a quantified conditioning bound. Formal coefficient construction can use multivariate graded jets or an ordered monomial set, but support order and precision become partial-order problems. The native implicit-asymptotic solver is a useful source of candidate solutions and differential tests, not an automatic certificate of a new package representation. [22]

A practical first target is a finite-dimensional real system near a regular point with a fixed weighted grading and a nonsingular exact leading Jacobian. Singular systems, several competing dominant balances, and parameter-dependent rank changes should remain separate milestones.

#### 14.5 Machine-checkable local evidence

The elementary interval certificate is particularly suitable for a small independent checker because its arithmetic endpoints are rational. A transcript could encode elementary expression nodes, input intervals, directed rounding witnesses, elementary tail bounds, derivative exclusion, and the final root-existence argument. Replay would validate a certificate without trusting the asymptotic planner or numerical seed generator.

For formal series identities, a second small checker could verify sparse coefficient arithmetic and the residual grade from a supplied model. Keeping these checkers separate respects the distinction between a formal identity, an analytic remainder theorem, and a numerical existence proof. A future proof-assistant integration could certify those small kernels before attempting the much larger symbolic-dispatch and simplification system.

### 15 Conclusion

The reviewed package has a coherent and valuable direction: explicit real scales, complete coefficient blocks, preserved remainder information, branch-aware inverse computation, and inspectable evidence. Its mathematical scope is already broader than its historical name suggests, and several important safeguards and performance improvements are already implemented.

The most actionable findings are at boundaries. Eager classical export can turn a tiny sparse answer into a huge allocation; that export also loses logarithmic tail information. A coefficient wrapper silently shadows an explicit option, and a generic refinement path does not always obtain the precision its caller requested. These issues are narrow enough to address without replacing the underlying theory.

The recommended strategy is to protect those boundaries, establish a current native release gate, and turn precision, capabilities, and evidence into stable interfaces. Native Wolfram functionality should remain both a backend and an independent comparison baseline wherever contracts overlap. Future progress should be judged not just by the number of recognizable functions, but by how clearly the system states what it knows, what it has assumed, what it has computed, and what it has actually proved.

## A Source coverage and reproducibility map

This appendix distinguishes direct source inspection from inventory and family-level assessment. It does not claim an exhaustive line-by-line audit of every repository file.

| Source or group            | Inspection scope                                               | Role in this review                                                                                                               |
|----------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Core AsymptoticInverse.wl  | Substantial selected ranges throughout                         | Input/exponent handling; sparse jets; forward/inverse assembly; exports; properties; residuals; coefficient API; module dispatch. |
| SeriesOperations.wl        | Substantial selected ranges                                    | Representation, arithmetic precision, differentiation, and generic refinement replay.                                             |
| SeriesArithmetic.wl        | Lines 1–165                                                    | Held normalization, regular operands, automatic arithmetic, and fallback classification.                                          |
| RefinementState.wl         | Lines 1–145                                                    | State signatures, prefix reuse, Newton state checks, result assembly, and precision ceilings.                                     |
| RefinementRequests.wl      | Complete short module                                          | Additional-block and numerical-certificate requests; confirms numeric-cutoff replay remains in the generic operation path.        |
| FourierCoefficients.wl     | Lines 1–310                                                    | Finite modes, realness, construction, coefficient envelopes, residuals, and coefficient queries.                                  |
| NativeSpecialFunctions.wl  | Lines 1–190                                                    | Native-series ingress, normalization, and integer-power implementation candidates.                                                |
| InverseCertificates.wl     | Lines 1–355, with the last retrieved range partially truncated | Rational interval arithmetic, domains, existence and uniqueness, options, and scope. Later adaptive-loop tail not fully audited.  |
| SpecialFunctionAdapters.wl | Lines 1–150                                                    | Erfc model bounds, explicit Gamma marker adapter, and threshold families.                                                         |
| SourceCoordinates.wl       | Lines 1–110                                                    | Log/exponential charts, exact reconstruction, domain handling, and explicit input-remainder limits.                               |
| GammaInverse.wl            | Lines 1–105                                                    | Family recognition, ordered core coefficients, and constructor checks.                                                            |



| Source or group                                                 | Inspection scope                                                       | Role in this review                                                         |
|-----------------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Other kernel modules                                            | Complete filename inventory; dispatch and current feature descriptions | Architectural and scope assessment, not an exhaustive implementation audit. |
| README, PacletInfo, license, CI, test runner, development notes | Directly retrieved text                                                | Public claims, packaging, migration, loading, and gates.                    |
| Current validation summary and historical review manifest       | Retrieved records, not native reruns                                   | Scope and provenance of reported tests.                                     |

The archive includes the full list of 38 kernel `.wl` module names and `init.m`. Its source-coverage JSON records the pinned revision and the retrieved ranges. Repository citations below link to that revision; official Wolfram references describe native capabilities rather than an independently measured installed kernel.

## B Artifact inventory and status

| Artifact                                                               | Purpose and validation status                                                                                |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>article/article.tex</code> ,<br><code>article/article.pdf</code> | This article and its compiled, visually checked PDF.                                                         |
| <code>code/independent_checks.py</code>                                | Executed independent mathematical checks; requires SymPy.                                                    |
| <code>code/AuditSupport.wl</code>                                      | Proposed nonintrusive Wolfram helpers; not executed natively.                                                |
| <code>patches/make_export_patch.py</code>                              | Source-pinned export containment patch generator; Python transformation tests passed.                        |
| <code>tests/test_python_tools.py</code>                                | Seven executed Python-tool tests.                                                                            |
| <code>tests/Baseline.wlt</code> ,<br><code>tests/Support.wlt</code>    | Unexecuted native baseline/helper tests.                                                                     |
| <code>tests/ProposedContracts.wlt</code>                               | Unexecuted desired contracts; not a claimed passing upstream suite.                                          |
| <code>tests/RunAudit.wls</code>                                        | Native runner with explicit suite selection, source file hash, and per-test result export.                   |
| <code>benchmarks/CompareBuiltins.wls</code>                            | Unexecuted native comparison harness; no automated unmatched speedup claims.                                 |
| <code>evidence/</code>                                                 | Executed independent results, native-execution status, source inventory, coverage, findings, and provenance. |
| <code>README.md</code> , <code>UPSTREAM-LICENSE.txt</code>             | Reproduction instructions, limitations, and the license for included upstream excerpts.                      |

## References

- [1] Vladimir Reshetnikov and Asymptotic contributors. Current package overview. `README.md`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/README.md>.
- [2] Vladimir Reshetnikov and Asymptotic contributors. Core implementation. `AsymptoticInverse/Kernel/AsymptoticInverse.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/AsymptoticInverse.wl>.
- [3] Vladimir Reshetnikov and Asymptotic contributors. Series operations and replay refinement. `AsymptoticInverse/Kernel/SeriesOperations.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesOperations.wl>.
- [4] Vladimir Reshetnikov and Asymptotic contributors. Automatic arithmetic and held normalization. `AsymptoticInverse/Kernel/SeriesArithmetic.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesArithmetic.wl>.
- [5] Vladimir Reshetnikov and Asymptotic contributors. Retained inverse computation state. `AsymptoticInverse/Kernel/RefinementState.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/RefinementState.wl>.
- [6] Vladimir Reshetnikov and Asymptotic contributors. Fourier coefficient calculus. `AsymptoticInverse/Kernel/FourierCoefficients.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/FourierCoefficients.wl>.
- [7] Vladimir Reshetnikov and Asymptotic contributors. Native special-function series boundary. `AsymptoticInverse/Kernel/NativeSpecialFunctions.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>.
- [8] Vladimir Reshetnikov and Asymptotic contributors. Real source-coordinate transformations. `AsymptoticInverse/Kernel/SourceCoordinates.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SourceCoordinates.wl>.
- [9] Vladimir Reshetnikov and Asymptotic contributors. Exact elementary inverse certificates. `AsymptoticInverse/Kernel/InverseCertificates.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseCertificates.wl>.
- [10] Vladimir Reshetnikov and Asymptotic contributors. Explicit special-function adapters. `AsymptoticInverse/Kernel/SpecialFunctionAdapters.wl`. Pinned revision `07a9781212be`; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SpecialFunctionAdapters.wl>.

- [11] Vladimir Reshetnikov and Asymptotic contributors. Ordered Gamma/Barnes inverse construction. `AsymptoticInverse/Kernel/GammaInverse.wl`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/GammaInverse.wl>.
- [12] Vladimir Reshetnikov and Asymptotic contributors. Paclet metadata. `AsymptoticInverse/PacletInfo.wl`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/PacletInfo.wl>.
- [13] Vladimir Reshetnikov and Asymptotic contributors. Current and historical validation record. `validation/README.md`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/validation/README.md>.
- [14] Vladimir Reshetnikov and Asymptotic contributors. Historical review manifest. `validation/review-manifest.json`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/validation/review-manifest.json>.
- [15] Vladimir Reshetnikov and Asymptotic contributors. Development provenance and standalone packaging. `docs/development/README.md`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/docs/development/README.md>.
- [16] Vladimir Reshetnikov and Asymptotic contributors. Standalone freshness CI workflow. `.github/workflows/standalone.yml`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/.github/workflows/standalone.yml>.
- [17] Vladimir Reshetnikov and Asymptotic contributors. Full-suite native runner. `AsymptoticInverse/Tests/RunTests.wl`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Tests/RunTests.wl>.
- [18] Vladimir Reshetnikov and Asymptotic contributors. Repository license text. `LICENSE`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/LICENSE>.
- [19] Vladimir Reshetnikov and Asymptotic contributors. Structured refinement requests. `AsymptoticInverse/Kernel/RefinementRequests.wl`. Pinned revision 07a9781212be; accessed 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/RefinementRequests.wl>.
- [20] Asymptotic repository tree at the reviewed revision. <https://github.com/VladimirReshetnikov/Asymptotic/tree/07a9781212beb2eeb9ff16aa625b50ac27974078>.
- [21] Wolfram Research. *Wolfram Language Documentation*, `Asymptotic`. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [22] Wolfram Research. *Wolfram Language Documentation*, `AsymptoticSolve`. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [23] Wolfram Research. *Wolfram Language Documentation*, `Series`. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.

- [24] Wolfram Research. *Wolfram Language Documentation*, **SeriesData**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [25] Wolfram Research. *Wolfram Language Documentation*, **InverseSeries**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [26] Wolfram Research. *Wolfram Language Documentation*, **ComposeSeries**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [27] Wolfram Research. *Wolfram Language Documentation*, **SeriesCoefficient**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesCoefficient.html>.
- [28] Wolfram Research. *Wolfram Language Documentation*, **ProductLog**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ProductLog.html>.
- [29] Wolfram Research. *Wolfram Language Documentation*, **AsymptoticIntegrate**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html>.
- [30] Wolfram Research. *Wolfram Language Documentation*, **DiscreteAsymptotic**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>.
- [31] Wolfram Research. *Wolfram Language Documentation*, **AsymptoticDSolveValue**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html>.
- [32] Wolfram Research. *Wolfram Language Documentation*, **AsymptoticLess**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticLess.html>.
- [33] Wolfram Research. *Wolfram Language Documentation*, **OptionValue**. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/OptionValue.html>.